MODULE 2.2 · Transformer Architecture

Self-Attention Mechanism

From the Q/K/V intuition to the full Transformer block — step by step

25 min	B3 – Apply	L2 – Foundational	Topics 1 & 2
Duration	Bloom's	Level	Builds on

Prof. Sashikumaar Ganesan · Indian Institute of Science, Bangalore

Learning Objectives

By the end of this session, you will be able to:

B2 – Explain

Why self-attention replaces recurrence and what bottleneck it solves

B2 – Define

The Query, Key, and Value matrices and their roles in the computation

B3 – Derive

The scaled dot-product formula step by step: QK^T / √dk, softmax, × V

B2 – Describe

How causal masking enforces left-to-right generation in decoder-only models

B3 – Distinguish

Multi-head attention from single-head and explain why h heads are used

B3 – Identify

All components of a full Transformer block: attention, FFN, LayerNorm, residual

Session Roadmap

6 sub-topics — motivation, mechanism, formula, masking, multi-head, full block

3.1	**Why Self-Attention? The Problem It Solves**	*Motivation*
3.2	**Query, Key, Value — The Library Analogy**	*Conceptual core*
3.3	**The Scaled Dot-Product Attention Formula**	*Step-by-step derivation*
3.4	**Causal Masking**	*Training vs generation*
3.5	**Multi-Head Attention**	*h heads in parallel*
3.6	**The Full Transformer Block**	*Attention + FFN + LayerNorm + Residual*

3.1

Why Self-Attention? The Problem It Solves

Before the mechanism — what was broken, and why recurrence had to go

The Sequential Bottleneck of RNNs

Token t cannot be processed until token t−1 is complete — parallelism is impossible

Sequential constraint: each token must wait for all previous tokens. No parallelism possible during training.

Sequential Bottleneck

Token t cannot be processed until token t−1 is done. Long sequences mean long serial chains. Training is slow.

Vanishing Gradients

Long-range dependencies are lost over many steps. The gradient signal fades before reaching early tokens.

Fixed Memory

A single hidden state vector must carry all prior context. Information from early tokens gets overwritten.

No Direct Token Links

To relate token 1 to token 100, information must flow through 99 intermediate hidden states.

Self-attention dissolves the sequential constraint: every token attends to every other token in a single parallel operation, regardless of distance.

Self-Attention — Direct Token-to-Token Links

Every token attends to every other token in one parallel operation

	The	animal	was	tired
The	**0.90**	0.30	0.20	0.10
animal	0.30	**0.85**	0.15	0.25
was	0.20	0.18	**0.88**	0.35
tired	0.10	0.22	0.30	**0.92**

Attention weights (rows sum to 1 after softmax)

RNN vs Self-Attention

Property	RNN	Self-Attn
Path length t1→tn	O(n)	O(1)
Training parallelism	None	Full
Long-range deps	Weak	Direct
Memory per step	Hidden state	Context window

Vaswani et al. (2017): Attention Is All You Need

3.2

Query, Key, Value — The Library Analogy

Intuition before formula: Q asks, K labels, V delivers

The Library Analogy

Before any matrix notation — build the intuition from a familiar scenario

Q Query

Library analogy:

The question you bring to the library

Technically:

A vector derived from the current token asking: what information do I need from other tokens?

Computed from the token embedding via learned matrix Wq

K Key

Library analogy:

The label on each shelf in the library

Technically:

A vector derived from each token advertising: what information do I contain?

Computed from the token embedding via learned matrix Wk

V Value

Library analogy:

The actual book content on each shelf

Technically:

A vector derived from each token holding: what information should I contribute if selected?

Computed from the token embedding via learned matrix Wv

Critical: Q, K, V are not retrieved from a fixed table — they are computed from the input via learned projections. They change with every input. There is no stored lookup.

Computing Q, K, V from Token Embeddings

Three learned projection matrices transform each token independently

For each token in the sequence, we compute three vectors:

$$Q = X \cdot W_q$$

Query matrix

X = token embeddings, Wq = learned (d_model × d_k)

$$K = X \cdot W_k$$

Key matrix

X = token embeddings, Wk = learned (d_model × d_k)

$$V = X \cdot W_v$$

Value matrix

X = token embeddings, Wv = learned (d_model × d_v)

Why three separate projections? Each one teaches the model a different question to ask about each token. The same word in different positions learns different Q, K, V vectors because its embedding changes with context.

3.3

The Scaled Dot-Product Attention Formula

Attention(Q,K,V) = softmax(QK^T / √dk) V — step by step

The Attention Formula — Four Operations

Each operation has a precise role. None can be removed.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

①	$Q \cdot K^T$	**Raw Attention Scores** *Each entry scores: how relevant is token j to token i?*	• Dot product measures similarity between each query and each key. • High dot product = high relevance. Result is an n×n score matrix for n tokens.
②	$\div \sqrt{dk}$	**Scale (Stabilise)** *This is the 'scaled' in Scaled Dot-Product Attention. Often dk = 64, so √dk = 8.*	• Divides every score by the square root of the key dimension dk. • Without this, large dk causes extremely peaked softmax distributions, vanishing gradients, and training instability.
③	$\text{softmax}(\cdot)$	**Normalise to Weights** *High weight = this token contributes heavily to the output. Zero weight = ignored.*	• Applies softmax row-wise. Converts raw scores to a probability distribution: all weights positive, each row sums to 1. • These are the attention weights.
④	$\times V$	**Weighted Sum of Values** *Output shape = same as input embedding dimension. Each token now carries context from all others.*	• Multiplies the attention weight matrix by the value matrix. The output for each token is a weighted average of all value vectors. • Tokens with high weights contribute more.

3-Token Worked Example — "The cat sat"

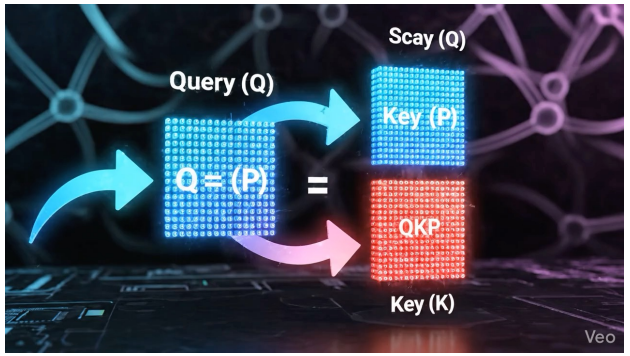
Let's walk through each operation by hand on a minimal sequence

Tokens: [The, cat, sat] | dk = 2 (simplified) | All values shown at 2 decimal places

Step ① Q and K matrices (X · Wq and X · Wk)

Q =	q_1	q_2
The	1.0	0.5
cat	0.8	0.9
sat	0.3	0.7

K =	k_1	k_2
The	0.9	0.4
cat	0.7	0.8
sat	0.2	0.6



Step ①② Scores = Q·K^T, then ÷ √2 = 1.414

Raw	The	cat	sat
The	1.10	1.16	0.50
cat	0.98	1.28	0.60
sat	0.55	0.77	0.48

Scaled	The	cat	sat
The	0.78	0.82	0.35
cat	0.69	0.91	0.42
sat	0.39	0.54	0.34

Step ③ Softmax row-wise → weights sum to 1.0 per row. Step ④ Multiply weights × V → context-enriched output vectors.

3-Token Worked Example — "The cat sat"

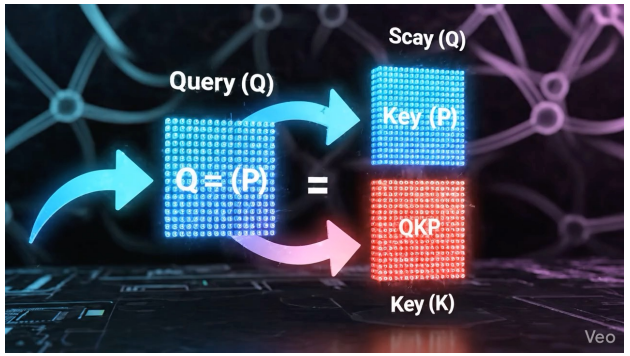
Let's walk through each operation by hand on a minimal sequence

Tokens: [The, cat, sat] | dk = 2 (simplified) | All values shown at 2 decimal places

Step ① Q and K matrices (X · Wq and X · Wk)

Q =	q_1	q_2
The	1.0	0.5
cat	0.8	0.9
sat	0.3	0.7

K =	k_1	k_2
The	0.9	0.4
cat	0.7	0.8
sat	0.2	0.6



Step ①② Scores = Q·K^T, then ÷ √2 = 1.414

Raw	The	cat	sat
The	1.10	1.16	0.50
cat	0.98	1.28	0.60
sat	0.55	0.77	0.48

Softmax(Scaled)	The	cat	sat
The	0.36	0.38	0.26
cat	0.31	0.40	0.29
sat	0.32	0.37	0.31

Step ③ Softmax row-wise → weights sum to 1.0 per row. Step ④ Multiply weights × V → context-enriched output vectors.

3.4

Causal Masking

Training is parallel. Generation is sequential. One architecture, two modes.

Causal Masking — Enforcing Left-to-Right Generation

Without masking, training would be trivially easy and the model would learn nothing

The problem: during training, the full target sequence is processed in parallel for efficiency. But if token 3 can see tokens 4, 5, 6... the prediction task is trivially solved by copying the next token. The model would learn nothing.

The causal mask — upper triangle set to −∞ before softmax:

Score	The	cat	sat	there
The	0.78	-inf	-inf	-inf
cat	0.69	0.91	-inf	-inf
sat	0.39	0.55	0.34	-inf
there	0.42	0.61	0.45	0.88

After softmax: −∞ → 0 weight. These positions contribute nothing to the output.

Training vs Inference

Training

Full sequence processed in parallel. Causal mask applied to prevent future token leakage. Efficient GPU utilisation.

Inference (Generation)

Tokens generated one at a time. No future tokens exist yet. Mask is implicit — not needed. Autoregressive loop from Topic 2.

Same architecture. Same weights. Two modes: parallel during training, sequential at inference. The deferred question from Topic 2 is now answered.

3.5

Multi-Head Attention

One head learns one relationship. h heads learn h perspectives in parallel.

Multi-Head Attention — h Perspectives in Parallel

Each head learns a different type of relationship between tokens

Limitation of single head:
One set of Q, K, V projections can only learn one type of relationship between tokens at a time. Language has many simultaneous structures: syntax, semantics, coreference, position.

Multi-head attention:
Run h attention heads in parallel, each with its own independent Wq, Wk, Wv projections. Concatenate the h outputs. Apply a final linear projection Wo.

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}h) \cdot Wo$$

$$\text{where} \quad \text{head}_i = \text{Attention}(\ Q \cdot Wq_i\ ,\ K \cdot Wk_i\ ,\ V \cdot Wv_i\ )$$

What different heads learn to attend to:

Head 1 — Syntactic dependencies

Subject–verb: "The cat that chased the dog sat" — cat attends to sat

Head 2 — Coreference resolution

"The animal didn't cross because it was tired" — it attends to animal

Head 3 — Positional proximity

Adjacent tokens attend strongly to their immediate neighbours

Head 4 — Semantic similarity

"bank" attends to financial terms or river terms based on context

Compute cost: dimension is split across heads (d_model / h per head), so total cost is roughly the same as one fat head. GPT-3 uses h=96; Llama 3 8B uses h=32.

3.6

The Full Transformer Block

Self-attention does not operate alone — four components, one block, stacked N times

The Full Transformer Block

Multi-Head Attention is one of four components — all four are required

1 Multi-Head Self-Attention

Attends to all other tokens — the mechanism from sub-topics 3.2–3.5

Runs h heads, concatenates, projects. Each token draws context from all positions.

2 Add & LayerNorm (Residual 1)

Residual connection + Layer Normalisation after attention

Residual: output = x + Attention(x). Prevents gradient vanishing in deep stacks. LayerNorm stabilises training by normalising activations.

3 Feed-Forward Network (FFN)

Applied independently to each token position — 2 linear layers + ReLU

Stores factual knowledge (~2/3 of model parameters). Expands to 4× hidden dim then contracts. Ablation studies show its removal causes severe performance degradation.

4 Add & LayerNorm (Residual 2)

Same pattern: output = x + FFN(x). Stacking N identical blocks creates progressively more abstract representations.

Residual connection + Layer Normalisation after FFN

Stack N of these blocks: GPT-3 uses N=96, Llama 3 8B uses N=32, Llama 3 70B uses N=80

The Feed-Forward Network — Not a Minor Component

~2/3 of model parameters live in FFN layers. This is where factual knowledge is stored.

FFN structure — two linear layers + activation

$$z = \text{ReLU}(x \cdot W_1 + b_1)$$

Expand to 4x d_model — e.g. 3072 → 12288 in GPT-3

$$\text{out} = z \cdot W_2 + b_2$$

Project back to d_model — e.g. 12288 → 3072

GPT-3: d_model=12288, FFN hidden=49152 (4×). W_1 shape: 12288×49152. Each block has ~2.4B FFN params.

Factual knowledge storage

Research shows that factual associations (Paris → capital of France) are stored as key-value pairs in FFN weights. Editing these weights changes what the model knows.

Per-position computation

The FFN is applied independently to each token position. It enriches each token's representation without mixing tokens together (unlike attention).

Ablation evidence

Removing FFN layers from a trained model causes severe performance degradation. Attention alone is insufficient for language understanding.

Assignment A3 — Attention Visualisation with BertViz

Make attention heads tangible — observe head specialisation with your own eyes

WHAT · WHY · EXPECTED OUTCOME

WHAT

Use BertViz to visualise attention heads on two sentences:

① "The animal didn't cross the street because it was too tired."

② "The animal didn't cross the street because it was too wide."

Identify which head(s) resolve the pronoun "it" in each case.

WHY

To make multi-head attention tangible. You will observe that different heads specialise in different relationships and that swapping one word shifts entire attention patterns across multiple heads.

Connects the library analogy and formula directly to a visible output.

EXPECTED OUTCOME

Two attention heatmaps — one per sentence.

A written answer (3–5 sentences) to: which head resolved coreference, what changed between the two sentences, and what that tells you about how the model represents meaning.

Bonus: identify a head that attends to syntactic structure vs one that attends to semantics.

pip install bertviz transformers · from bertviz import head_view · Submit notebook before mentor session

Four Misconceptions — Cleared

Common errors that arise at this stage of the curriculum

M1

MYTH: "Self-attention looks up a dictionary"

CORRECTION: Attention scores are computed from learned projections, not retrieved from a fixed lookup table. They change with every input — the same word produces different Q, K, V vectors in different contexts.

M2

MYTH: "Transformers process sequences left-to-right"

CORRECTION: The encoder is fully bidirectional — all tokens see all others simultaneously. Only decoder generation is left-to-right, enforced by causal masking at inference. Training is always parallel.

M3

MYTH: "Self-attention replaces all computation in a Transformer"

CORRECTION: Each Transformer block also contains a Feed-Forward Network (FFN), Layer Normalisation, and Residual Connections. The FFN alone accounts for ~2/3 of model parameters. Attention alone is insufficient.

M4

MYTH: "The $\sqrt{d_k}$ scaling is unimportant"

CORRECTION: Without it, large d_k causes extremely peaked softmax distributions and vanishing gradients, significantly harming training stability. The scaling is mathematically necessary, not cosmetic.

References & Further Reading

Foundational papers, tools, and resources for Topic 3

Foundational Papers

[1] **Vaswani et al. (2017). Attention Is All You Need. NeurIPS.**
Original Transformer paper — defines scaled dot-product and multi-head attention

[2] **Bahdanau et al. (2015). Neural Machine Translation by Jointly Learning to Align and Translate.**
Precursor to Transformer attention — additive attention in RNN seq2seq

[3] **Meng et al. (2022). Locating and Editing Factual Associations in GPT. NeurIPS.**
Demonstrates FFN layers store factual associations as key-value memories

[4] **Elhage et al. (2021). A Mathematical Framework for Transformer Circuits. Anthropic.**
Mechanistic interpretability — how information flows through attention heads

[5] **Dao et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention. NeurIPS.**
Efficient attention implementation — unlocks long context at inference

Tools & Visualisations

Tool	**BertViz — Interactive attention head visualisation** github.com/jessevig/bertviz
Blog	**Jay Alammar — The Illustrated Transformer** jalammar.github.io/illustrated-transformer
Tool	**Tensor2Tensor — Attention visualiser by Google** colab.research.google.com/github/tensorflow/tensor2tensor
Video	**Andrej Karpathy — Attention mechanism lecture** youtube.com/watch?v=kCc8FmEb1nY
Tool	**TransformerLens — Mechanistic interpretability** github.com/neelnanda-io/TransformerLens
Paper	**Anthropic — Transformer Circuits Framework** transformer-circuits.pub

Topic 3 Summary — Self-Attention Mechanism

Six essential takeaways

1

Self-attention solves the RNN bottleneck

Every token attends to every other token in O(1) path length, in a single parallel operation. Vanishing gradients and sequential constraints are eliminated.

2

Q asks, K labels, V delivers

Q, K, V are computed from token embeddings via learned projections. They change with every input. There is no fixed lookup table.

3

The formula: softmax($QK^T/\sqrt{dk}$)V

Four operations in sequence: dot product (similarity), scale (stability), softmax (normalise), weighted sum (aggregate). The $\sqrt{dk}$ prevents softmax saturation.

4

Causal masking answers the Topic 2 question

Training is parallel (mask applied). Inference is sequential (future tokens don't exist). Same architecture, same weights, two modes.

5

h heads learn h perspectives

Multi-head attention runs h independent attention operations. Different heads specialise: syntax, coreference, position, semantics. Total cost ~= one fat head.

6

Self-attention is one of four block components

FFN stores factual knowledge (~2/3 of parameters). LayerNorm stabilises training. Residual connections allow deep stacks. All four are required.

Topics 1, 2 & 3 — What Carries Forward

The stack so far — and what unlocks next

Topic 1 Completed	Topic 2 Completed	Topic 3 Completed	Topic 4 Next
✓ Token vocabulary	✓ Autoregressive loop	✓ RNN bottleneck	○ ViT patch tokens
✓ Softmax + logits	✓ Stopping conditions	✓ Q / K / V projections	○ Image embeddings
✓ Temperature	✓ Repetition failure	✓ Attention formula	○ VLM architecture
✓ Sampling strategies	✓ 3 hallucination types	✓ Causal masking	○ CLIP contrastive
✓ Cross-entropy	✓ Confidence ≠ truth	✓ Multi-head attn	○ Shared embed space
✓ Perplexity	✓ 3 misconceptions	✓ Full Transformer block	○ Bridge to RAG

Completing Topic 3 unlocks: LoRA (M7) · KV Cache (M3) · Flash Attention (M9) · Cross-Attention in RAG (M5) · Full pre-training pipeline (M6)

What We Deferred — and Where It Lands

Deliberate deferrals: each item has a more appropriate home in the curriculum

M3 — LLM APIs Recap	**Context Window & KV Cache**	*Requires knowledge of the full Transformer block (now complete) and API cost implications*
M9 — Inference Optimisation	**Flash Attention**	*Exact attention with memory-efficient IO — requires understanding of standard attention first (now complete)*
M5 — RAG & Retrieval	**Cross-Attention (Encoder-Decoder)**	*Decoder attending to encoder output — relevant when studying seq2seq and RAG architectures*
M2.4 — Positional Encoding	**Positional Encoding Derivation**	*Sinusoidal and rotary (RoPE) formulations — detailed derivation warrants its own concept session*
M2.6 — Decoder-Only Architecture	**Encoder vs Decoder Architecture Deep Dive**	*Bidirectional vs causal, BERT vs GPT — requires the full block understanding now established*
M7 — Fine-Tuning & PEFT	**LoRA and Parameter-Efficient Fine-Tuning**	*Low-rank adaptation of Q/K/V weight matrices — directly uses the projection matrices from Topic 3*

Topic 3 Complete

Self-Attention Mechanism

Coming up next in this session:

Topic 4	**Multimodal AI & Vision Transformers (M4 Section 1)** Images as token sequences. ViT patch decomposition, image embeddings, VLM architecture, CLIP contrastive learning, shared embedding space.
M3 Recap	**LLM APIs — Context Window, KV Cache, Latency/Cost** Now that the full Transformer block is understood, the API-level implications of context length and KV cache can be covered properly.
A3	**Assignment A3 — BertViz Attention Visualisation** Observe head specialisation on ambiguous coreference sentences. Submit two heatmaps and written analysis before mentor session.

Assignment A3 issued · Submit before mentor session next week · Questions during the break